Singly Linked List – Delete At A Position And Time Complexity.

Program:

```
void deleteAtPosition(Node **head, int position)
{
    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return;
    }
    if (position < 1)
    {
        cout << "Error: Invalid position." << endl;
        return;
    }
    else
    {
        Node *current = *head;
        Node *previous = nullptr;
        int i = 1;
        while (i < position && current != nullptr)
        {
            previous = current;
            current = current->next;
            i++;
        }
        if (current == nullptr)
        {
            cout << "Error: Invalid position." << endl;
            return;
        }
```

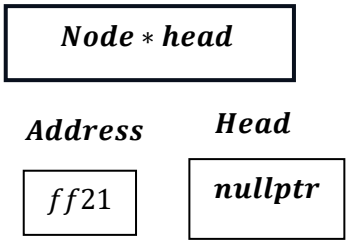
```
        else
        {
            if (previous == nullptr)
            {
                *head = current->next;
            }
            else
            {
                previous->next = current->next;
            }
          cout << "Deleted node: " << current->data << endl;
          current->next = nullptr;
          free(current);
         current = nullptr;
          previous = nullptr;
          cout << "Node is deleted successfully." << endl;
        }
    }
}

…. .

   case 8:
        {
            cout << "Delete at a position,indexing node from 1 to N:" << endl;
            int position;
            cout << "Enter the node position to delete: ";
            cin >> position;
            deleteAtPosition(&head, position);
            break;
        }
```

Program Analysis:

There are two criteria of input Linked List:

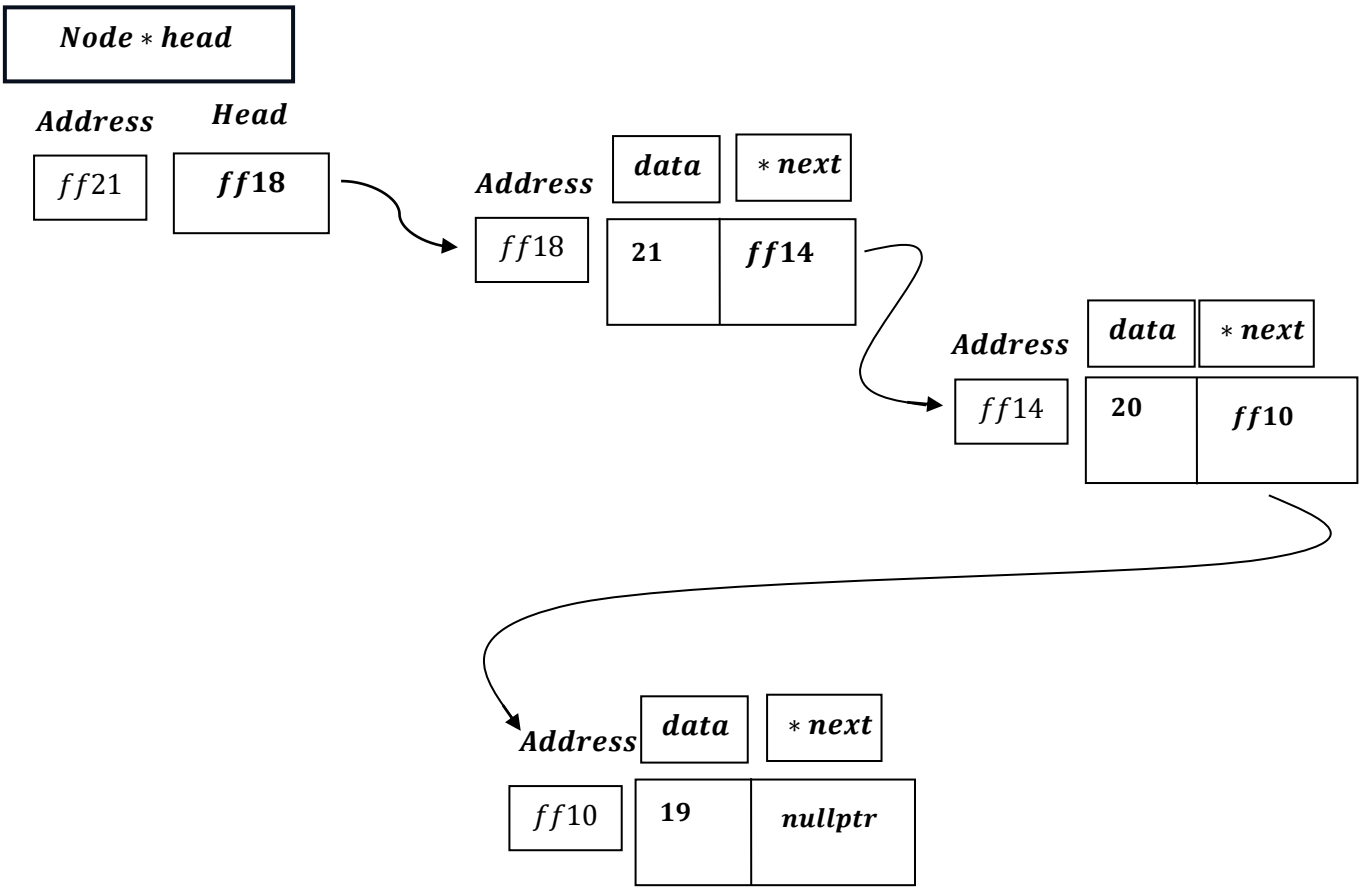
1)***Linked List is Empty***:

Pictorially depicted as:



2)***Linked List is not Empty***:

Pictorially depicted as:

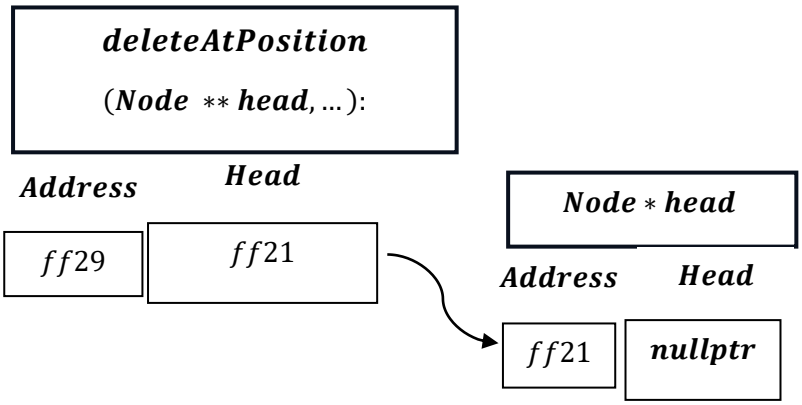


1) *When Linked List is Empty*:

```
            deleteAtPosition(&head, position);
```

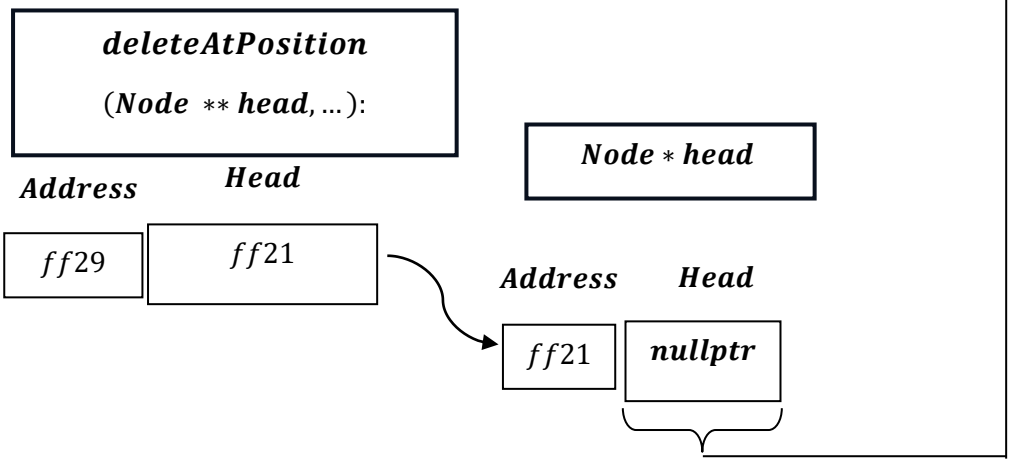
$deleteAtPosition(ff21, 1);$

```
void deleteAtPosition(Node **head, int position) {…}
```



$if(*head == nullptr)$ *if it is true*

If true then Why true ?
*Ans: Though * head which is of Node ** head*
temporary pointer to pointer variable of the function
*, as * head is used , it dereference from Node ** head and*
*refer to Node * head global pointer to which Node ** head*
is pointing to and then its value is checked is
whether equal to nullptr or not ? This occurs during execution
and compilation of the deleteAtPosition function.



```
    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return;
    }
```

If List is Empty , then , it prints a message : `Error: List is empty.` Here input position entered by user doesn't matters.

---

2) *When Position is less than* 1

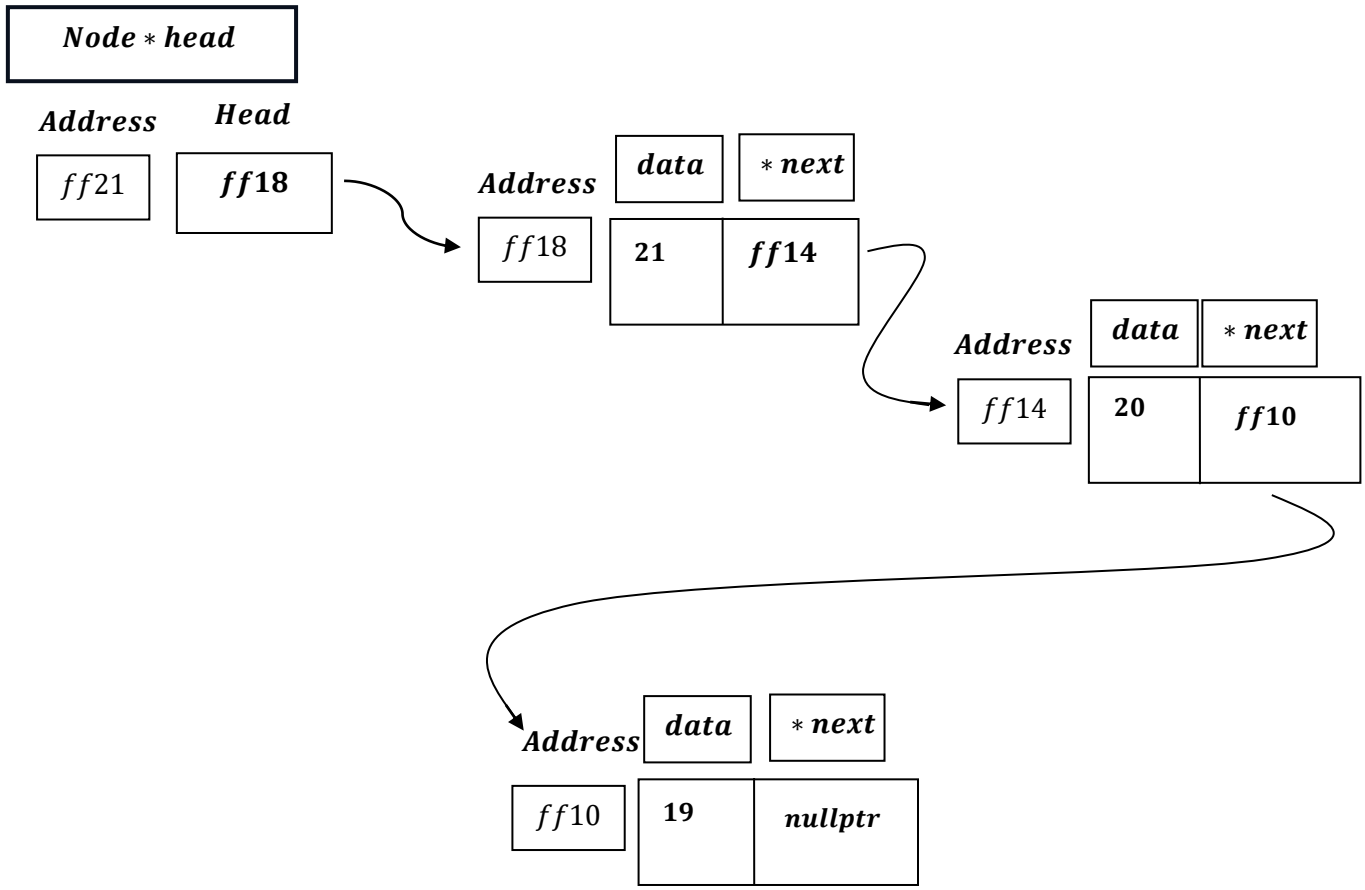
Here index position starts from 1, *deletion ends in* N^{th} *position.*

```
    if (position < 1)
    {
        cout << "Error: Invalid position." << endl;
        return;
    }
```

Hence if user input position is less than 1 *, then then it returns message : ` Error : Invalid position.`*

---

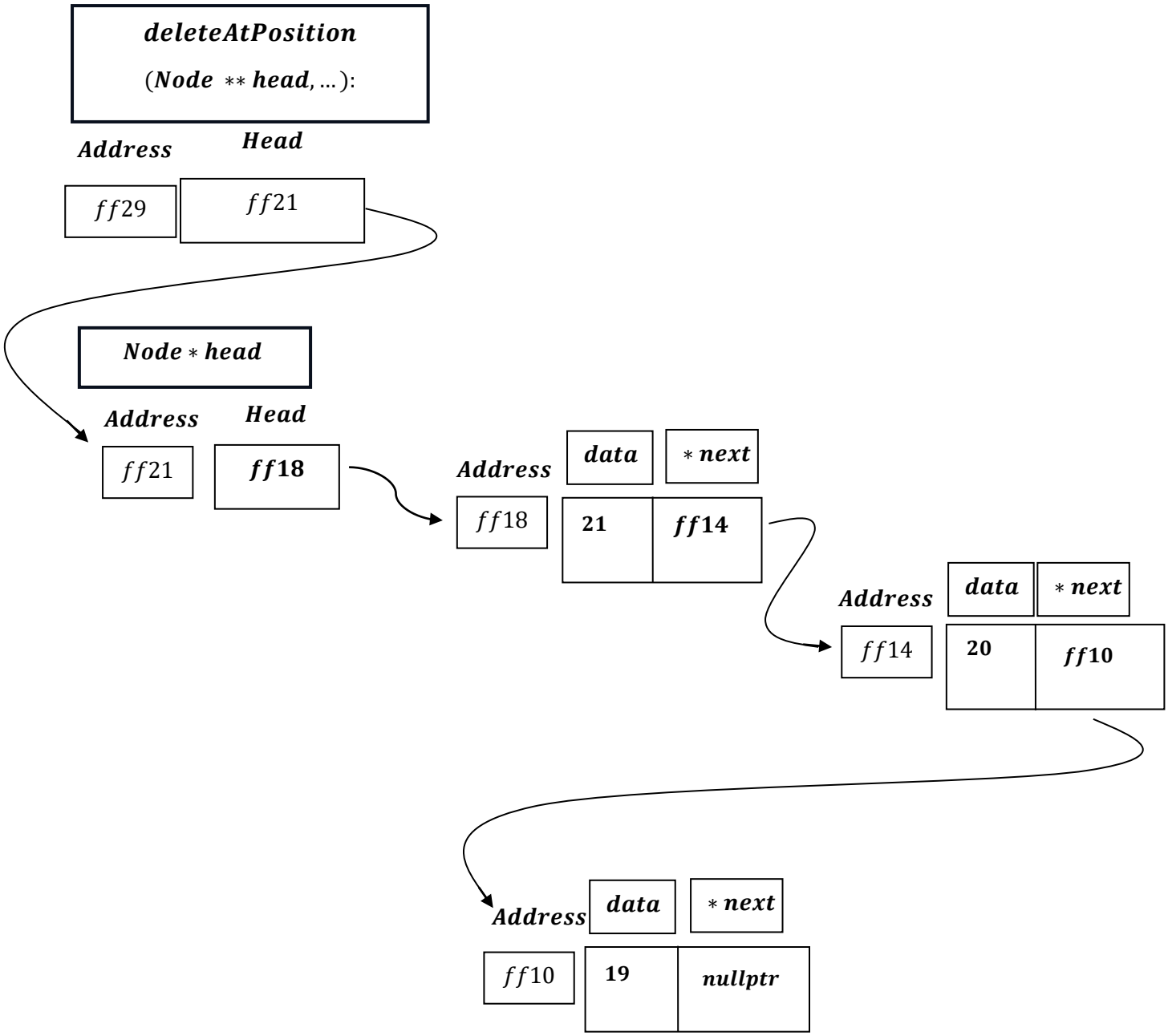
3) *When Linked List is not Empty And Position is not less than* 1:




```
deleteAtPosition(&head, position);
```

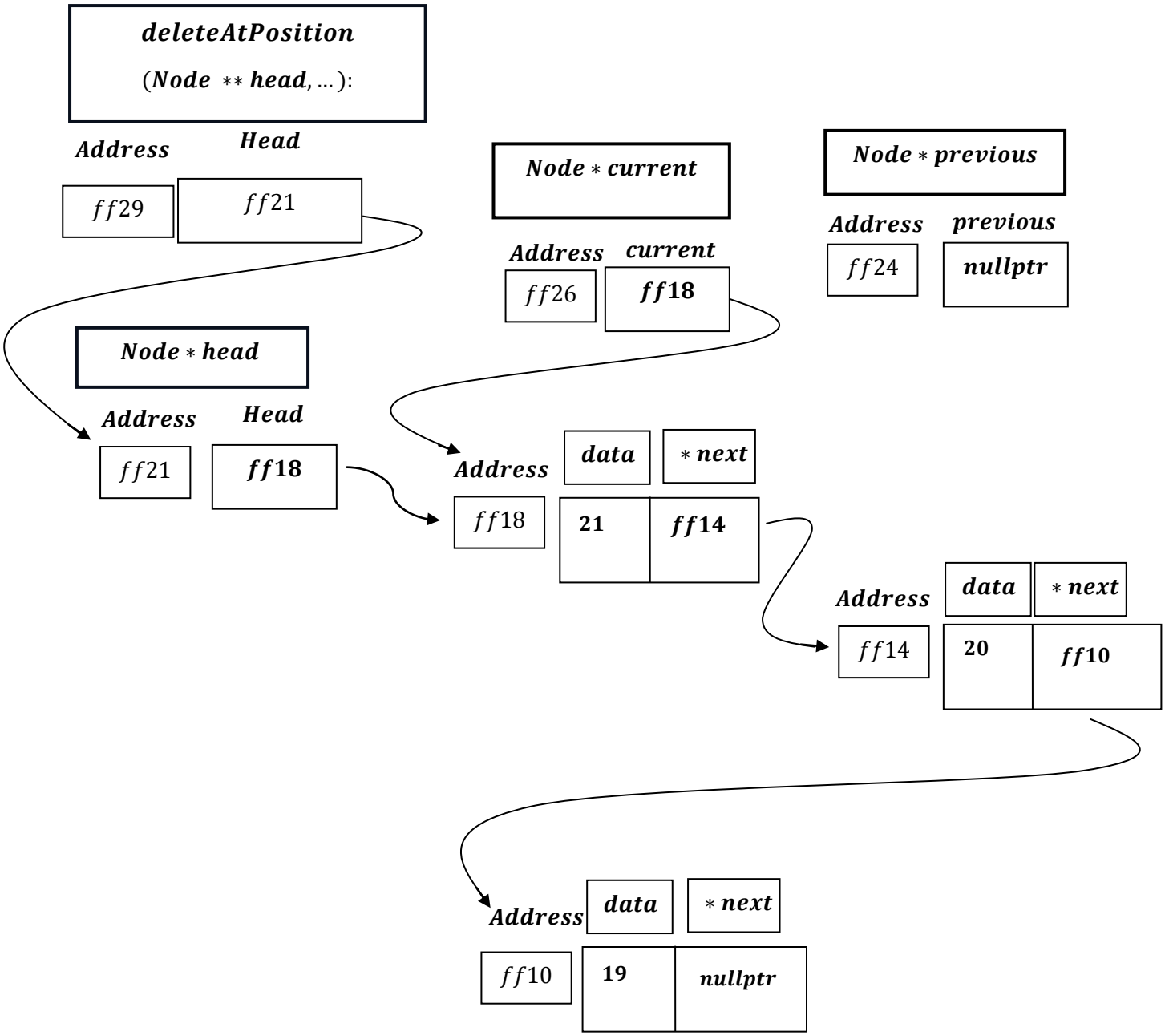
$deleteAtPosition(ff21, 2)$; *As length of the linked list is* 3. *And we have to delete* 2nd *node*.

```
void deleteAtPosition(Node **head, int position) {…}
```



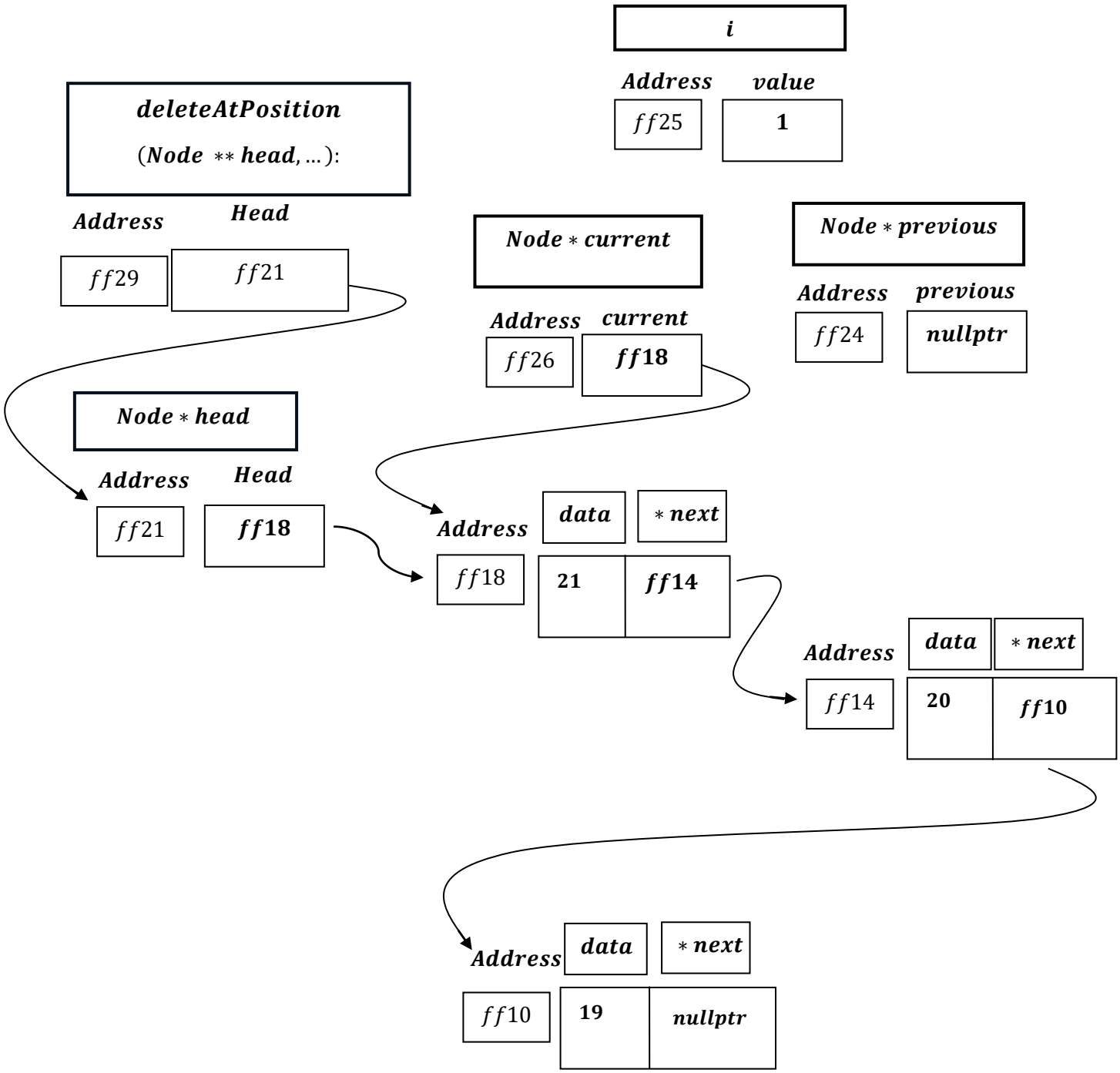
Next:

```
        Node *current = *head;
        Node *previous = nullptr;
```


deleteAtPosition
(Node ** head, ...):
Address
Head
ff29
ff21
Node * current
Address
current
ff26
ff18
Node * previous
Address
previous
ff24
nullptr
Node * head
Address
Head
ff21
ff18
Address
data
* next
ff18
21
ff14
Address
data
* next
ff14
20
ff10
Address
data
* next
ff10
19
nullptr

```
int i = 1;
```

$As, i = 1,$




```
        while (i < position && current != nullptr)
        {
            previous = current;
            current = current->next;
            i++;
        }
```

Our position is : 2, hence , total number of iteration needed is 1

$\left[i.e. Total\ number\ of\ nodes\ less\ than\ Head\ Node\ , as\ current\ already\ pointing\ to\ Head\ Node.\right]$:

$i < position\ i.e. i < 2$, $hence\ i\ will\ looped\ for\ once(1)\ only\ and\ at\ this$ $rate$, $current\ will\ never\ become\ nullptr\ as\ current \rightarrow next\ will\ not\ reach$ $the\ end\ node\ which\ contain\ next\ pointer\ value:\ nullptr$, if $position\ is\ between\ 1\ to\ N\ i.e. suppose\ length\ of\ linked\ list\ is\ 8$, $then$ $node\ can\ be\ deleted\ at\ position: 1, 2, \ldots., 8\ and\ current\ will\ never$ $carry\ `nullptr`$.

$But\ if\ position\ is\ 9\ current\ carries\ Nullptr, and\ loop\ will\ get\ terminated$.

$Now\ our\ length\ of\ the\ linked\ list\ is: 3\ and\ we\ want\ to\ delete\ data\ at\ node$ $position: 2\ i.e. at\ n - 1\ node.$

$when\ i = 1\ < 2\ and\ current\ is\ not\ equal\ to\ `nullptr`:$

```
previous = current;
current = current->next;
i++;
```

$i.e. current\ pointer's\ value\ is\ assigned\ to\ the\ value\ contained\ in$ $current \rightarrow\ next, i.e. address\ of\ next\ node.$

$And\ previous\ pointer\ of\ Node\ structure\ assigned\ to\ current\ pointer's\ value\ of$ $Node\ structure.$

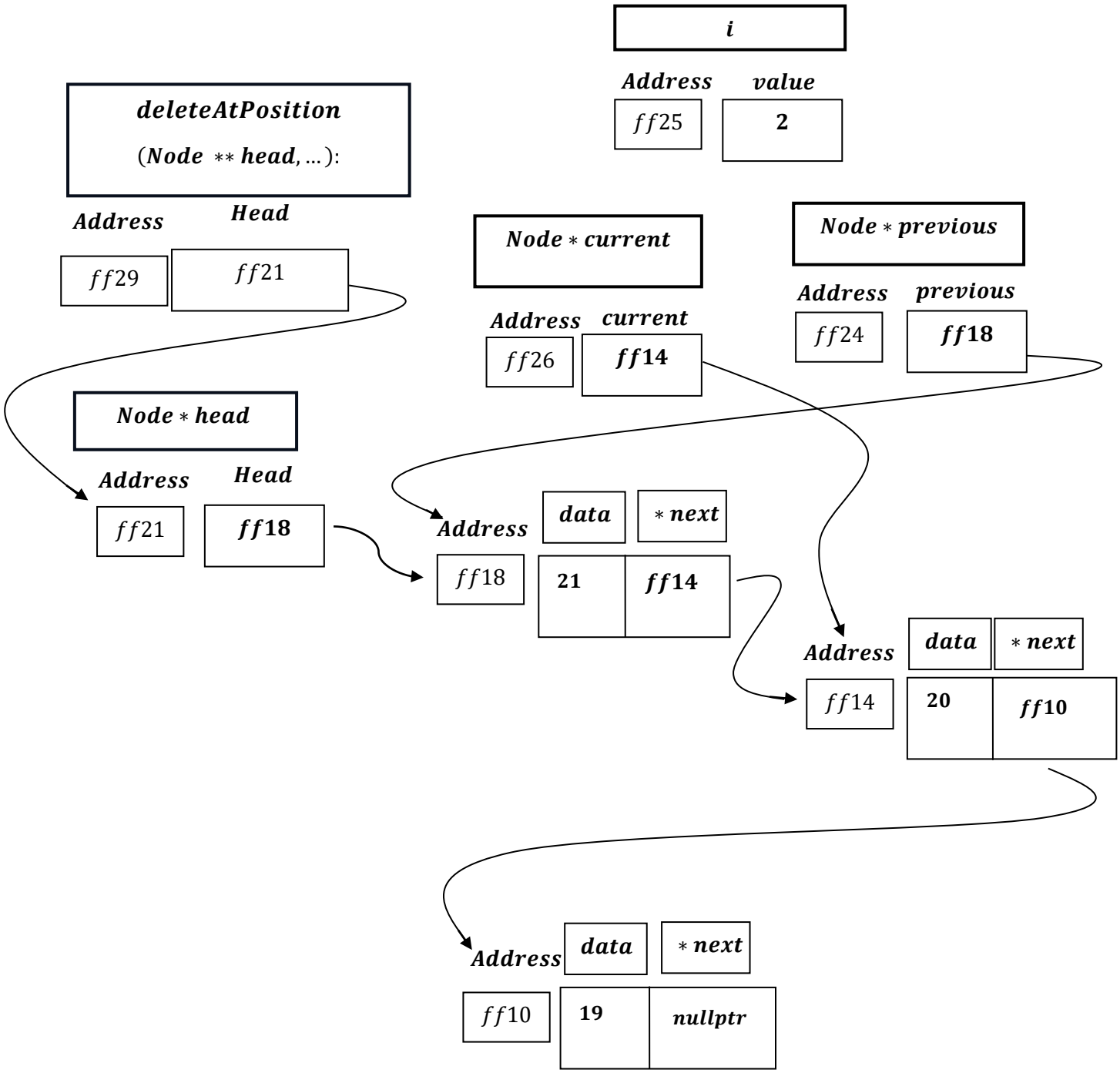
$And\ i\ gets\ updated\ each\ time.$

$previous = current = ff18;$

$current = current \rightarrow next = ff14;$

$\boldsymbol{i} + += \boldsymbol{i} + \mathbf{1} = \mathbf{2}$.

pictorially represented as:



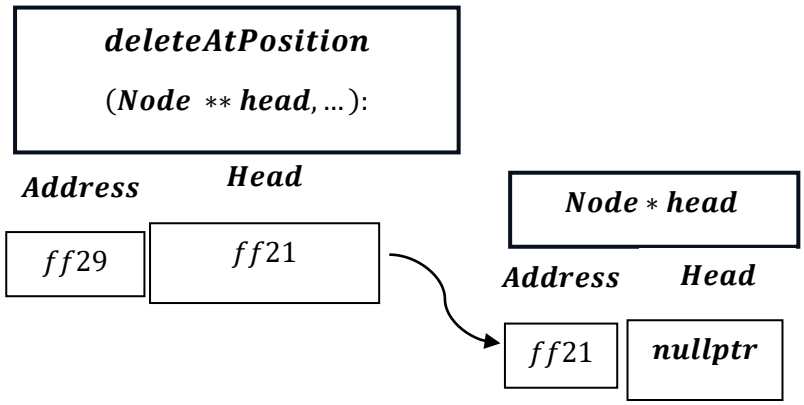
Next,

$when\ (i = 2\ < 2\ is\ false\ and\ current\ is\ not\ equal\ to\ `nullptr`\ is\ true) is\ false\ due\ to\ 0\ AND\ 1 = 0\ i.e. FALSE\ AND\ TRUE = FALSE, as\ both\ the\ statement\ must\ be\ true\ to\ get\ result\ true\ for\ anding.$

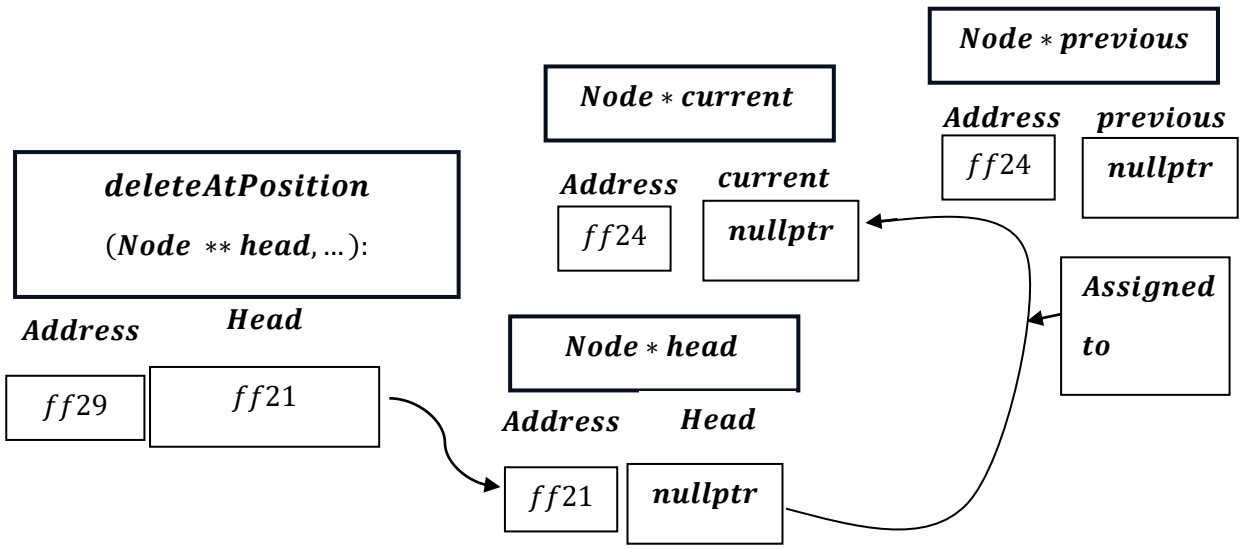
```
        if (current == nullptr)
        {
            cout << "Error: Invalid position." << endl;
            return;
        }
```

$When\ current\ is\ nullptr\ ?\ The\ time\ when\ head\ pointer's\ value\ is\ `nullptr`\ and\ when\ position\ is\ out\ of\ bounds.$

1) *When head pointer's value is nullptr*



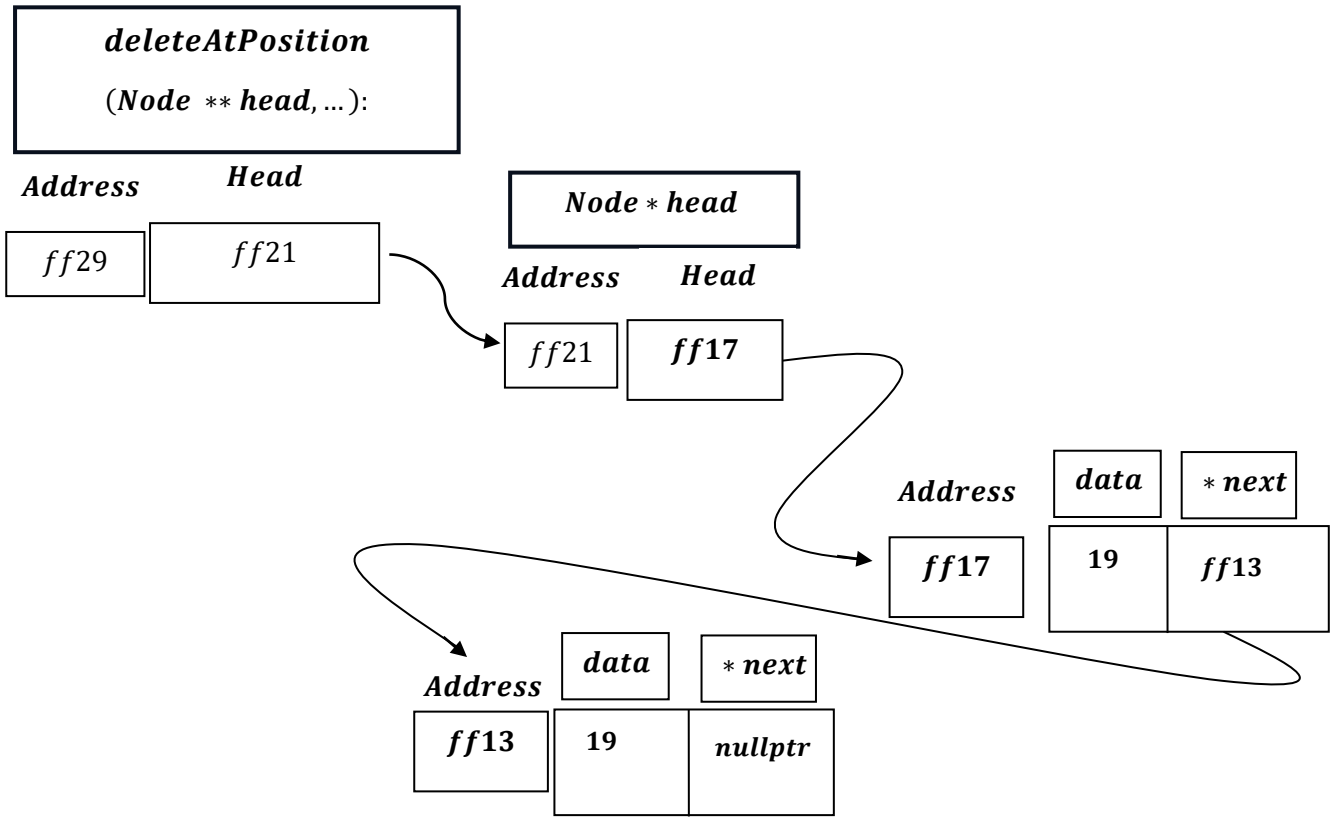
$And\ , Node * current = * head; and\ Node * previous = nullptr;$



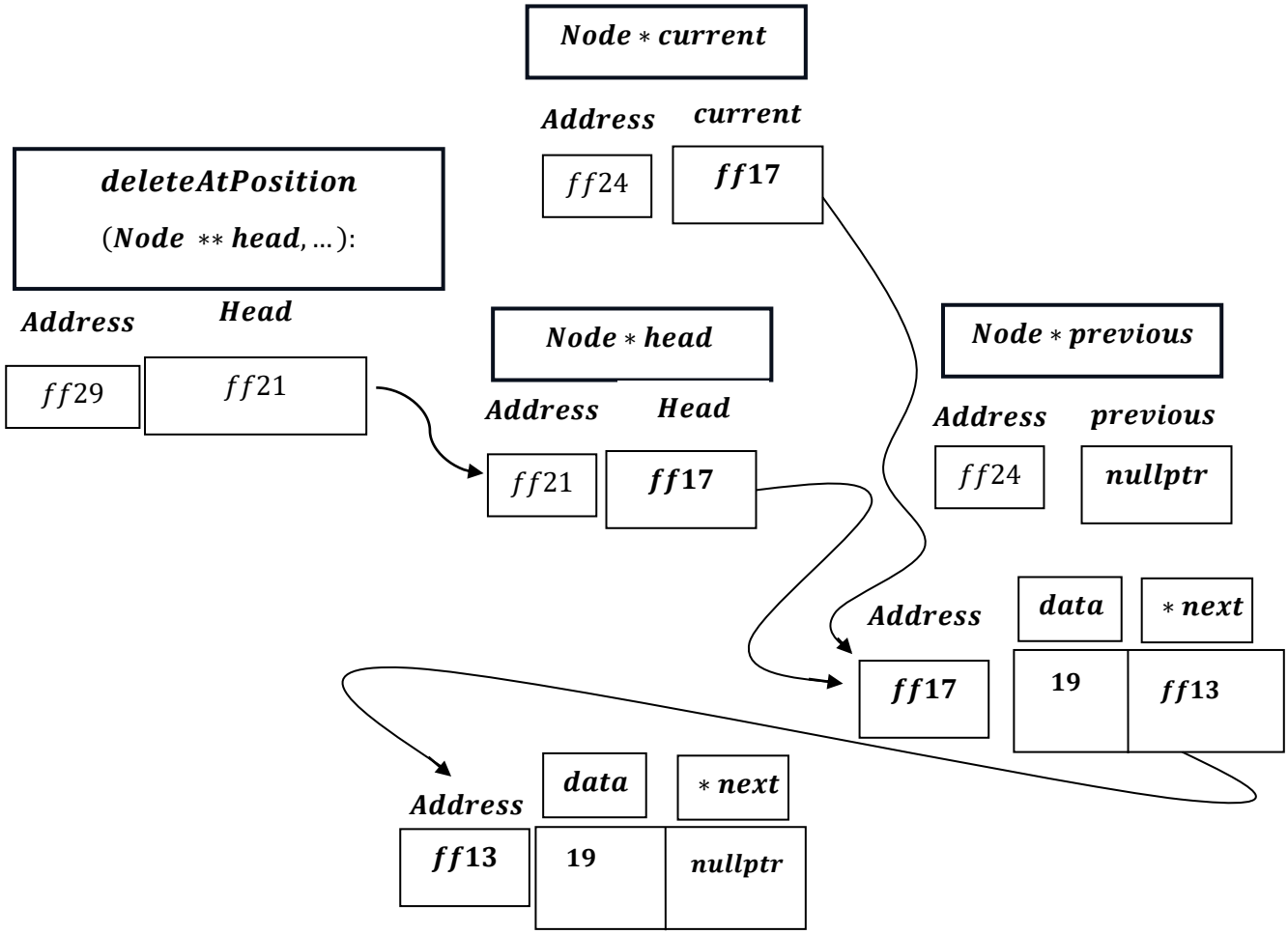
Note , this type of situation never arrive as we already have(At starting of function):

```
if (*head == nullptr)
  {
      cout << "Error: List is empty." << endl;
      return;
  }
```

2. *When Position Is Out Of Bounds*



At this type of situation, when Node * current = * head; and Node * previous = nullptr;:



Here length of the Linked List is : **2** [***Node*** **1** ***and Node*** **2**]

Now, we pass function call: ***deleteAtPosition***(***ff*****21**, **4**). [***As Node*** **4** ***is unavailable***]

```
        while (i < position && current != nullptr)
        {
            previous = current;
            current = current->next;
            i++;
        }
```

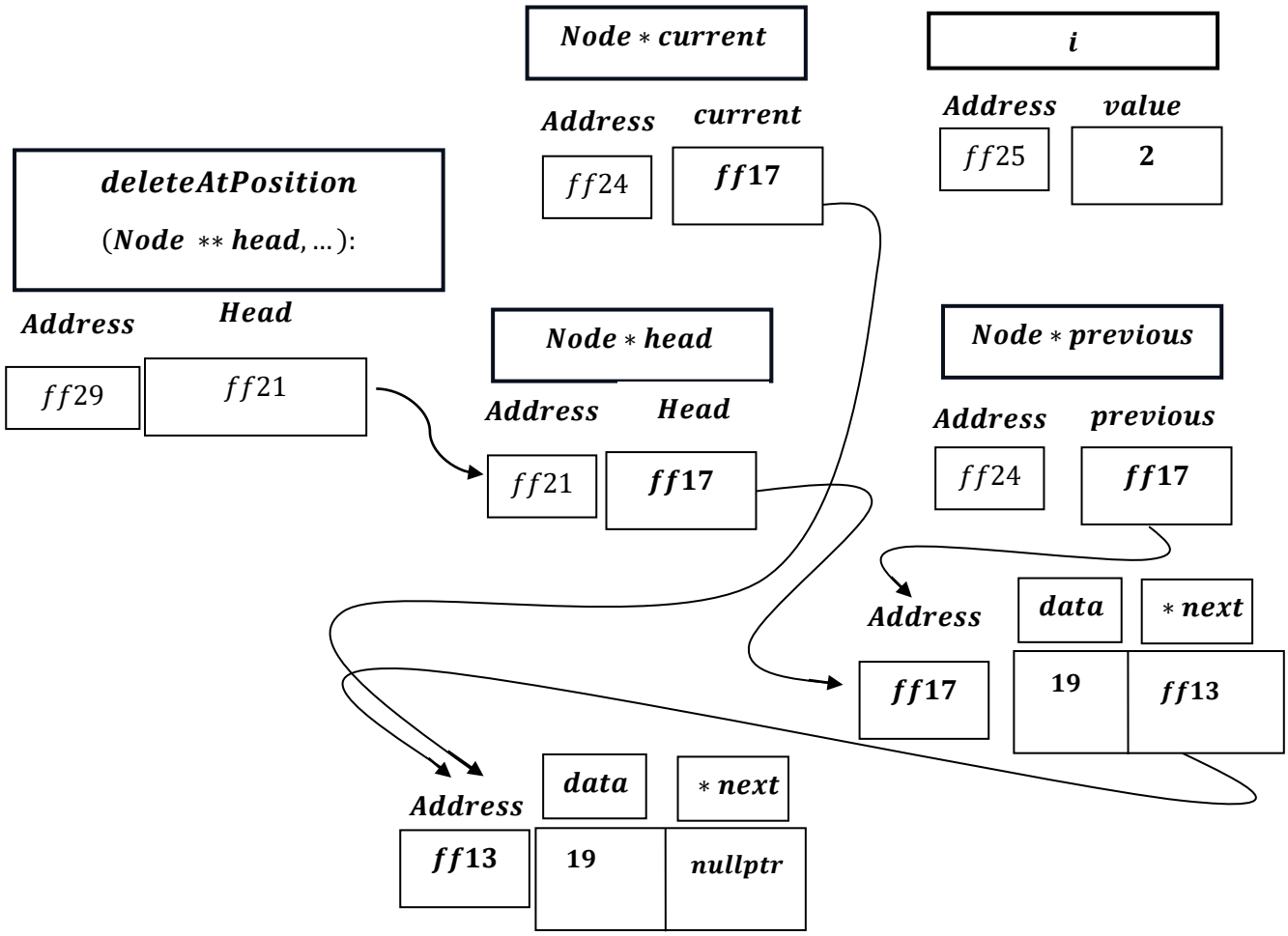
$i = 1$.

when $i = 1 < 4$ ***and current is not equal to*** `nullptr` :

$previous = current = ff17.$

$current = current \rightarrow next = ff13.$

$i += i + 1 = 2.$

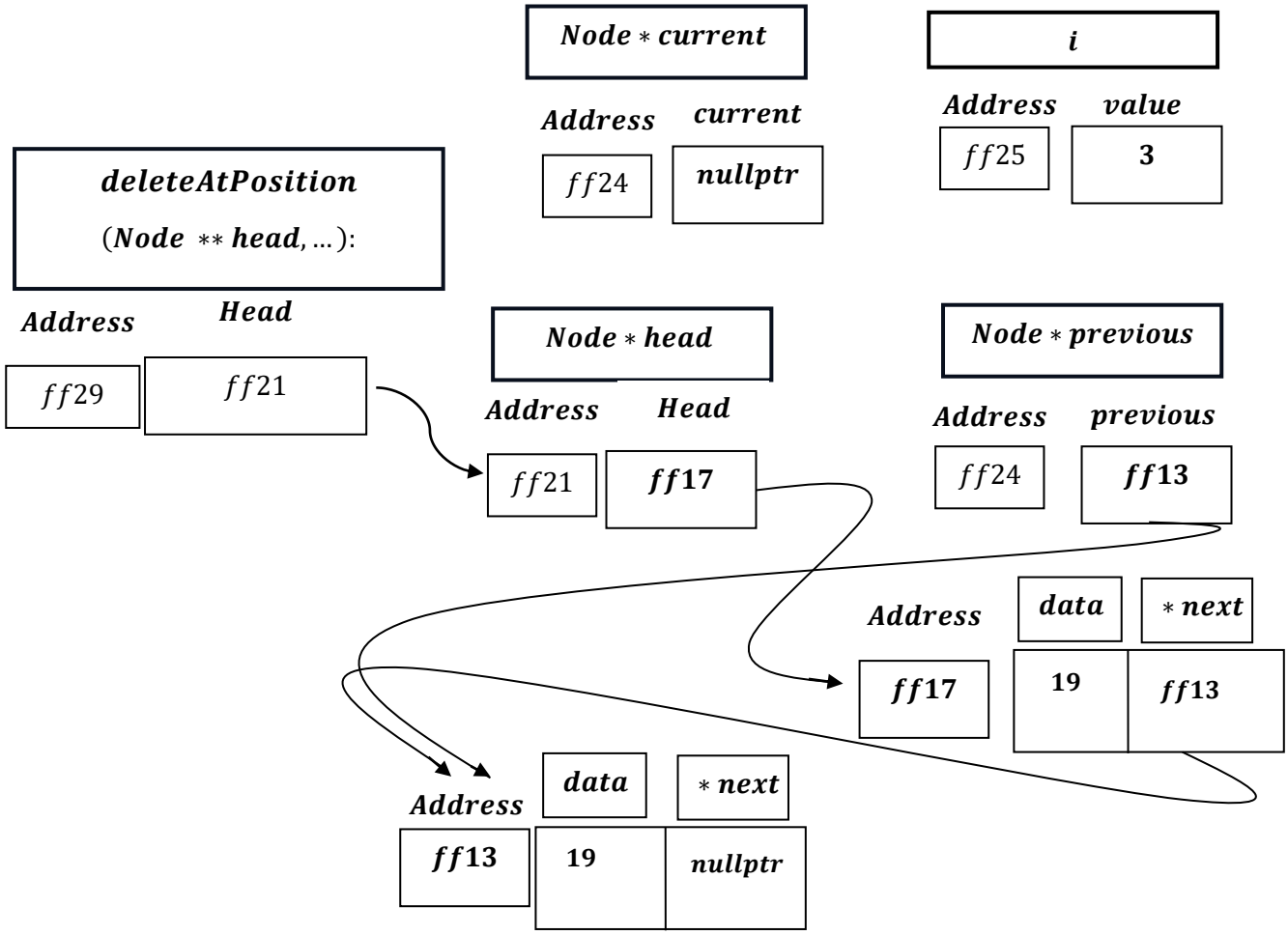


$when\ i = 2\ < 4\ and\ current\ is\ not\ equal\ to\ `nullptr`:$

$previous = current = ff13.$

$current = current \rightarrow next = nullptr.$

$i += i + 1 = 3.$



Then, when $i = 3 < 4$ (is true) and current is not equal to `nullptr` (is False) :

when ($i = 3 < 4$ is True and current is not equal to `nullptr` is False) is false due to 1 AND 0 = 0 i.e. TRUE AND FALSE = FALSE, as both the statement must be true to get result true for anding.

Hence, now current is nullptr , when position is out of bounds, therefore:

```
        if (current == nullptr)
        {
            cout << "Error: Invalid position." << endl;
            return;
        }
```

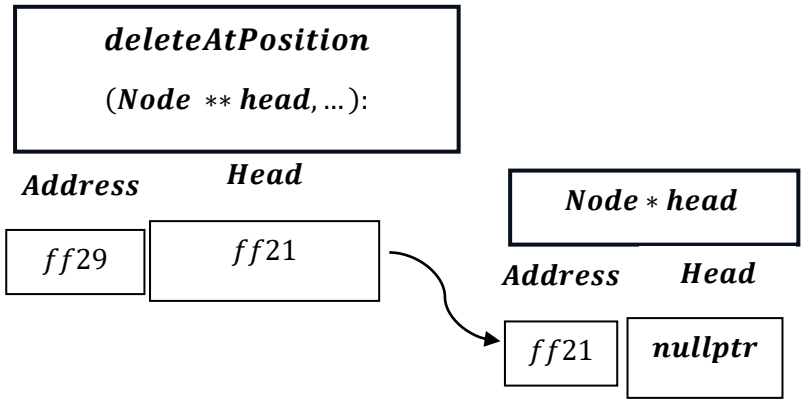
It prints an error message : "Invalid Position" and ends the function.

Now, we move towards else statement:

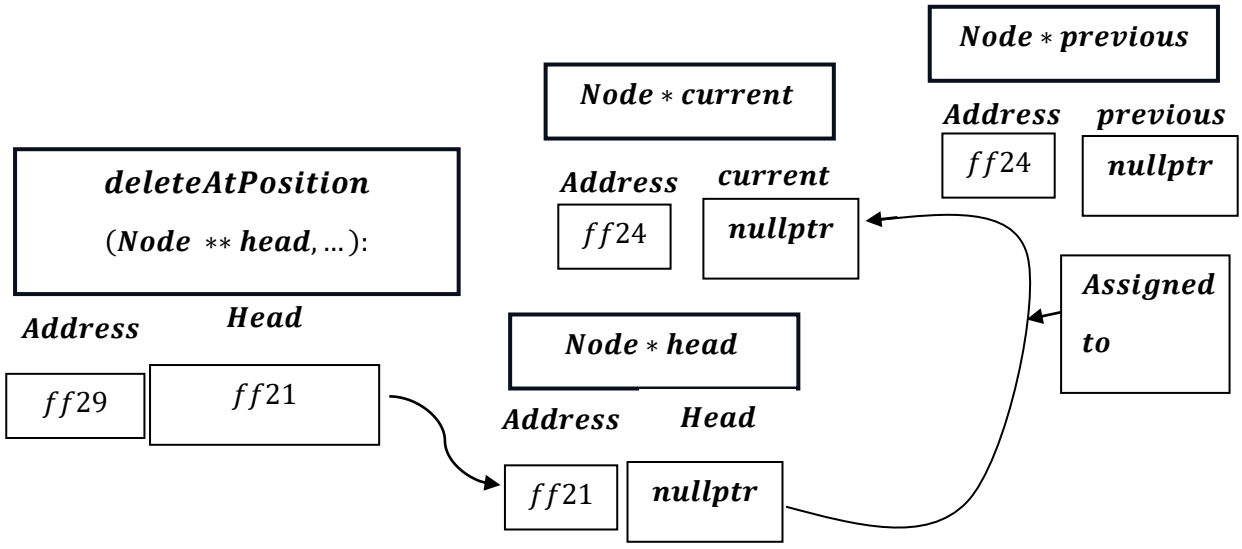
```
else
        {
            if (previous == nullptr)
            {

                *head = current->next;
            }
            else
            {

                previous->next = current->next;
            }
….
}
```

Now , when previous remains at nullptr:

It remains on two condition, first condition is when head is nullptr logically it is possible as shown below:



*And , Node * current = * head; and Node * previous = nullptr;*



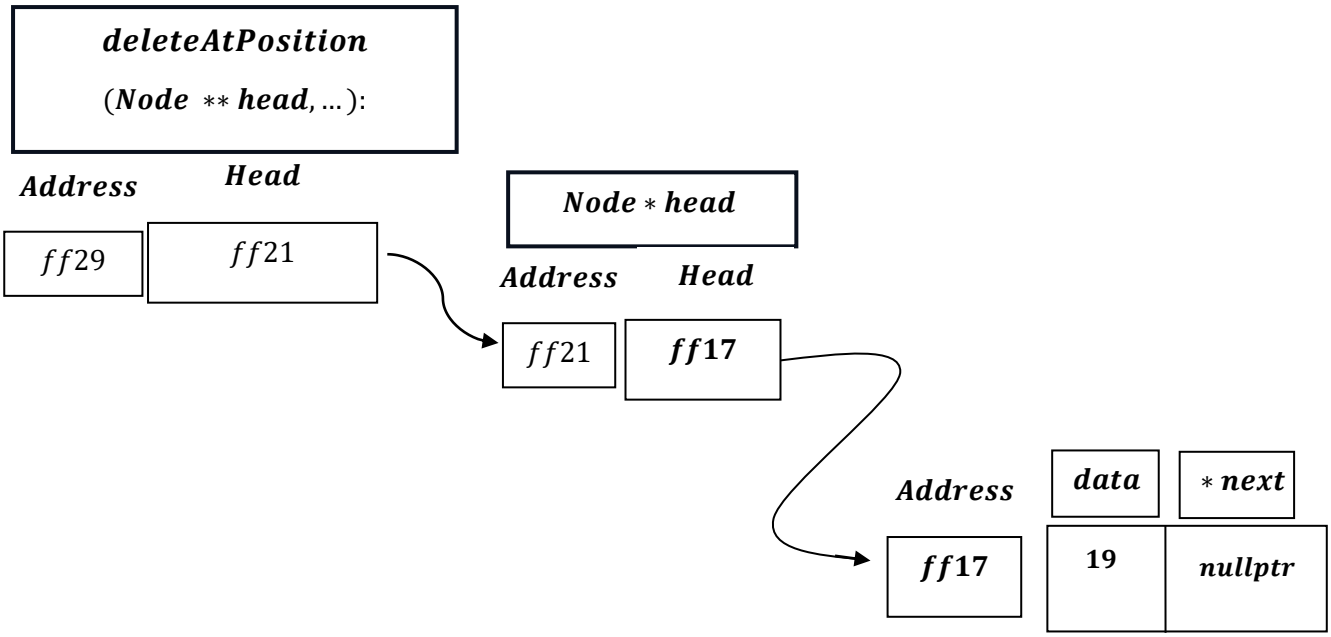
But practically it is not possible due to the condition earlier:

```
if (*head == nullptr)
  {
      cout << "Error: List is empty." << endl;
      return;
  }
```

<u>When head is nullptr , at the beginning of the function it checks and returns with an error message , hence no possibility of this situtation.</u>

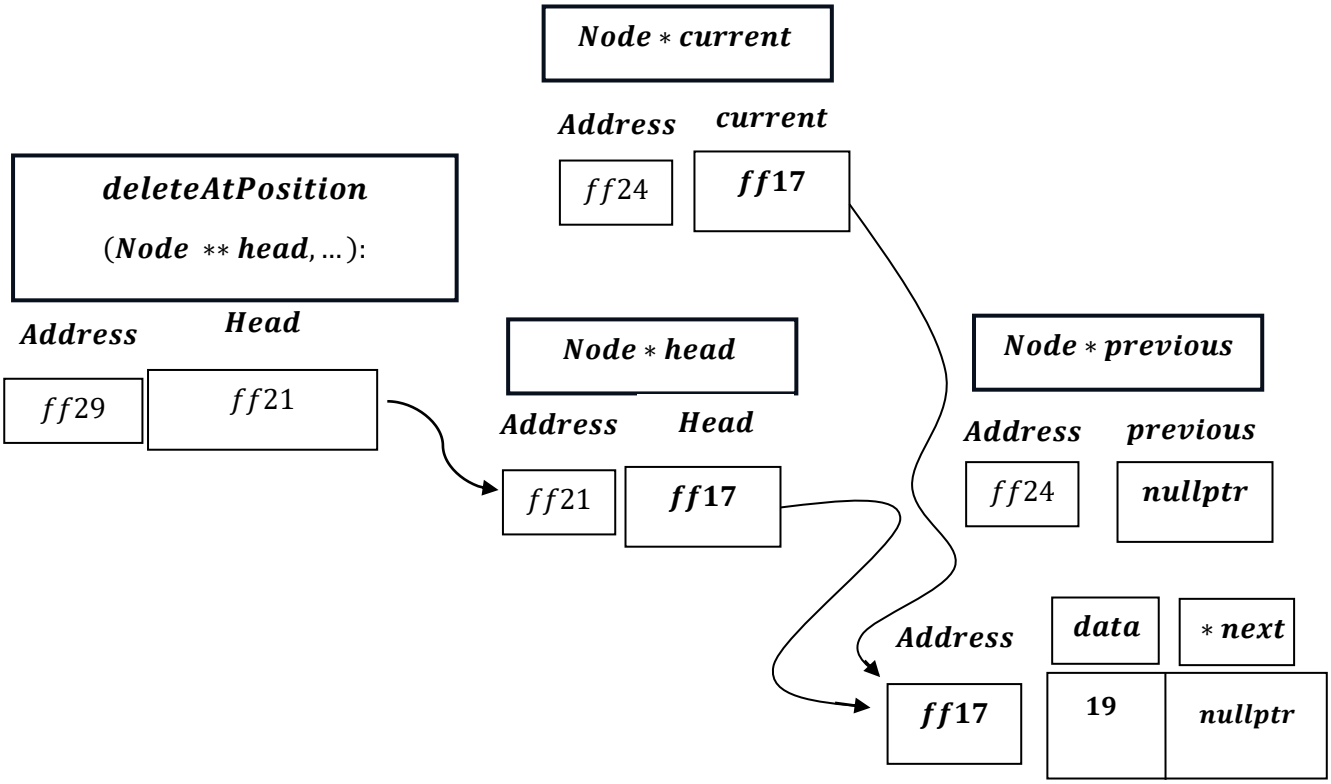
Second possibility is , when there is single node attached to head pointer:

i. e.



At this type of situation, when Node * current = * head; and

Node * previous = nullptr;:



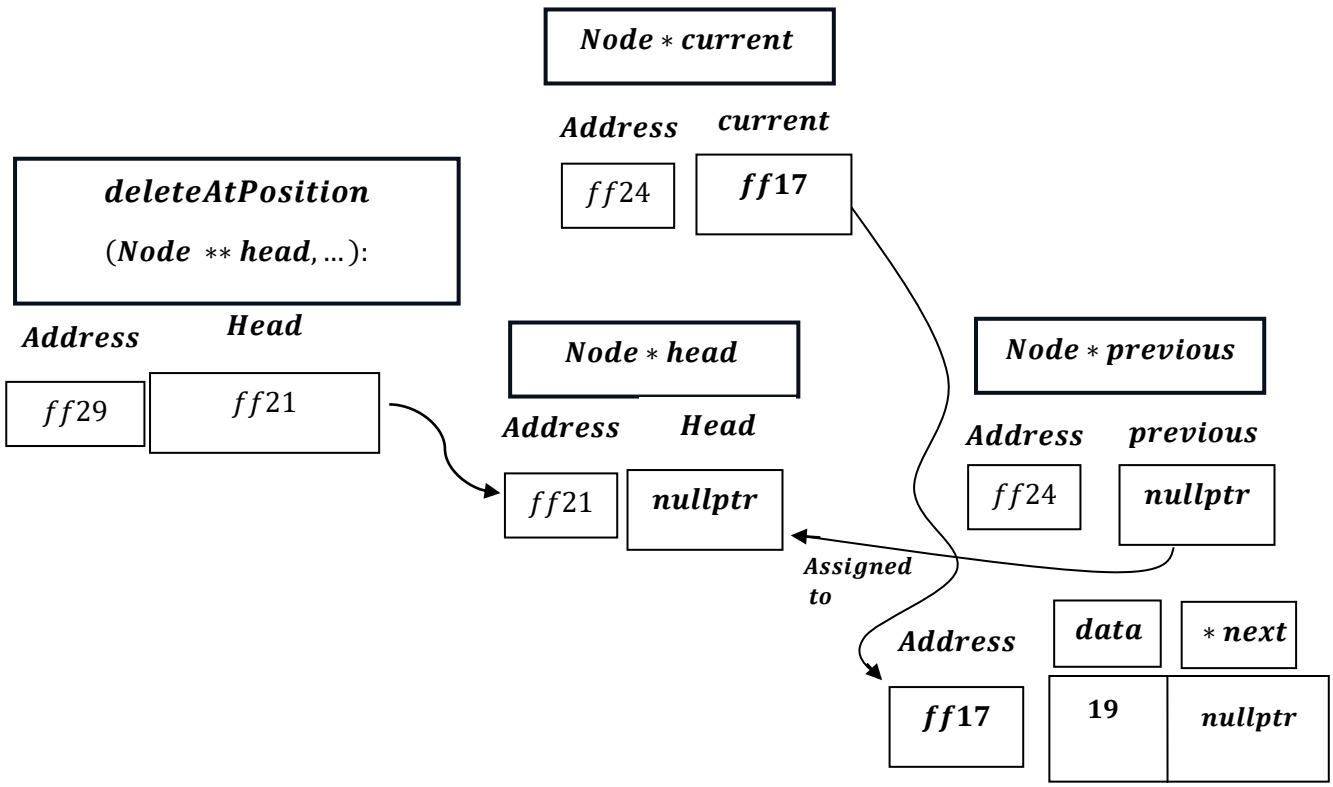
Now , length of the node is only 1 and loop will run $i = 1 < 1$ which is false and loop execution fails at first run as condition checking fails , hence loop never going to run .

This make , previous pointer's to remain `nullptr` .

Hence, here we can do it in two ways:

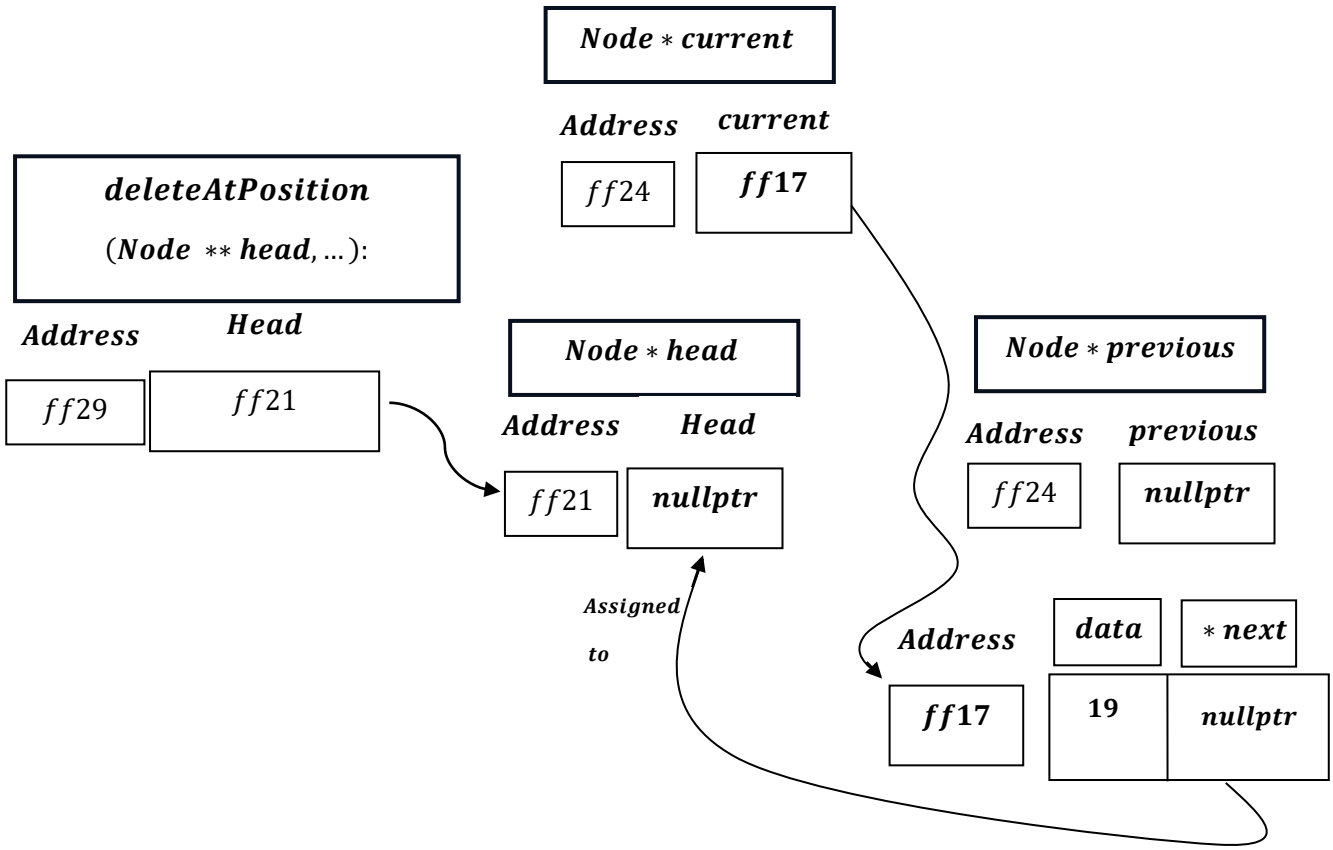
1st way is $*head = previous$[Hence,$*head = previous = nullptr$]:

```
if (previous == nullptr)
            {

                *head = previous;
            }
```



$2nd\ way\ is * head = current \rightarrow next[Hence, * head = current \rightarrow next = nullptr\ ]$:

```
if (previous == nullptr)
            {
                *head = current->next;

            }
```

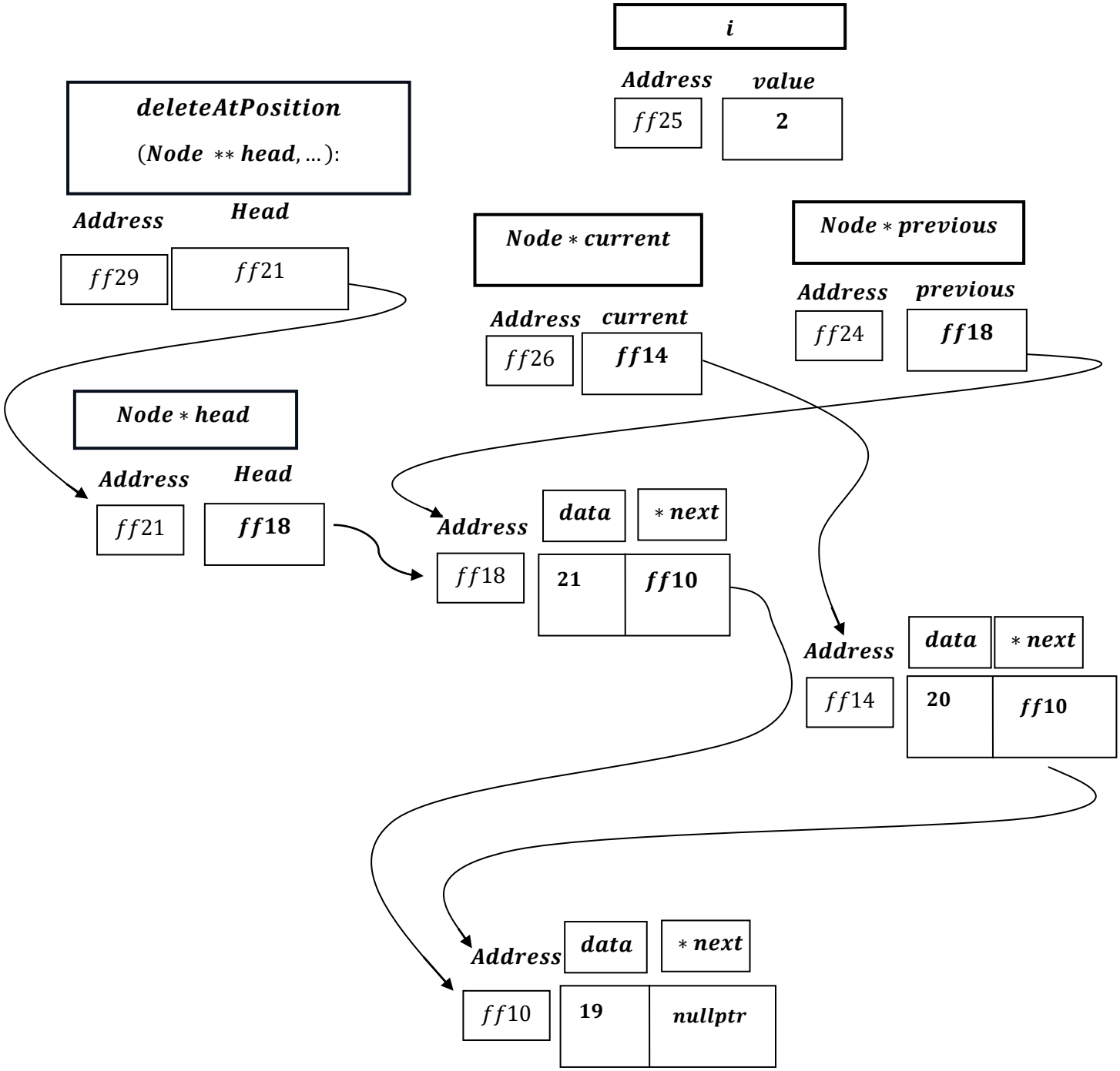


$Now, we\ have\ multiple\ nodes\ in\ Linked\ List\ , hence$:

```
if (previous == nullptr){…}
else{
        previous->next = current->next;
    }
```

$i.e. previous \rightarrow next = current \rightarrow next = ff10.$

[*Recalling earlier example when, length of the list is 3, and we delete position 2*
i.e. 2nd Position, when we have nodes: Node 1, Node 2, Node 3, [Indexing from 1 to 3]].

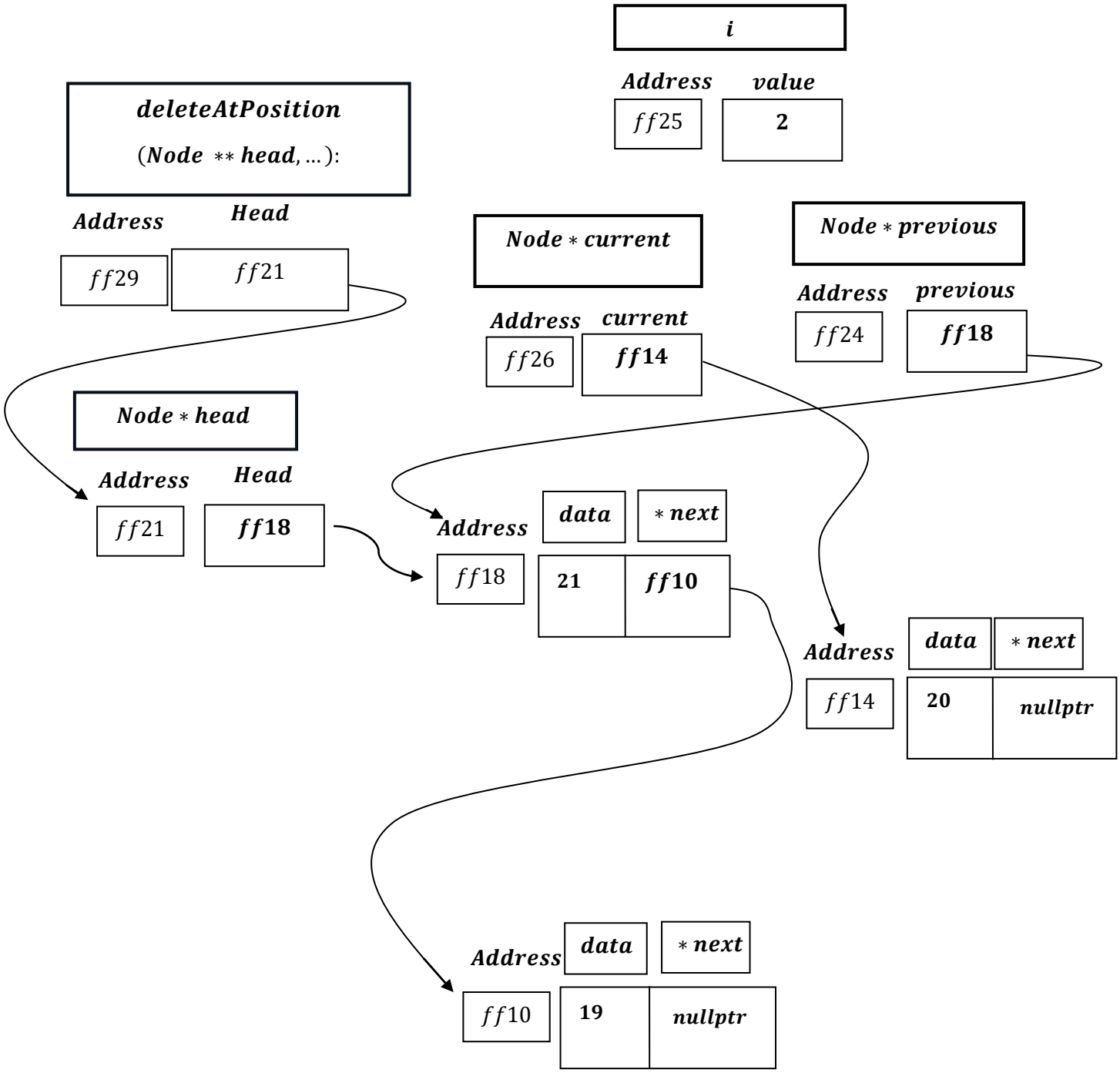



```
cout << "Deleted node: " << current->data << endl;
```

Next, we print out the data that contains in the node which is to be deleted.

```
current->next = nullptr;
```

$i.e. current \rightarrow next = nullptr$; *which makes next pointer stop to point to address which, the pointer next previously pointing* . [*Here*, $ff10$ *to NULLPTR*].

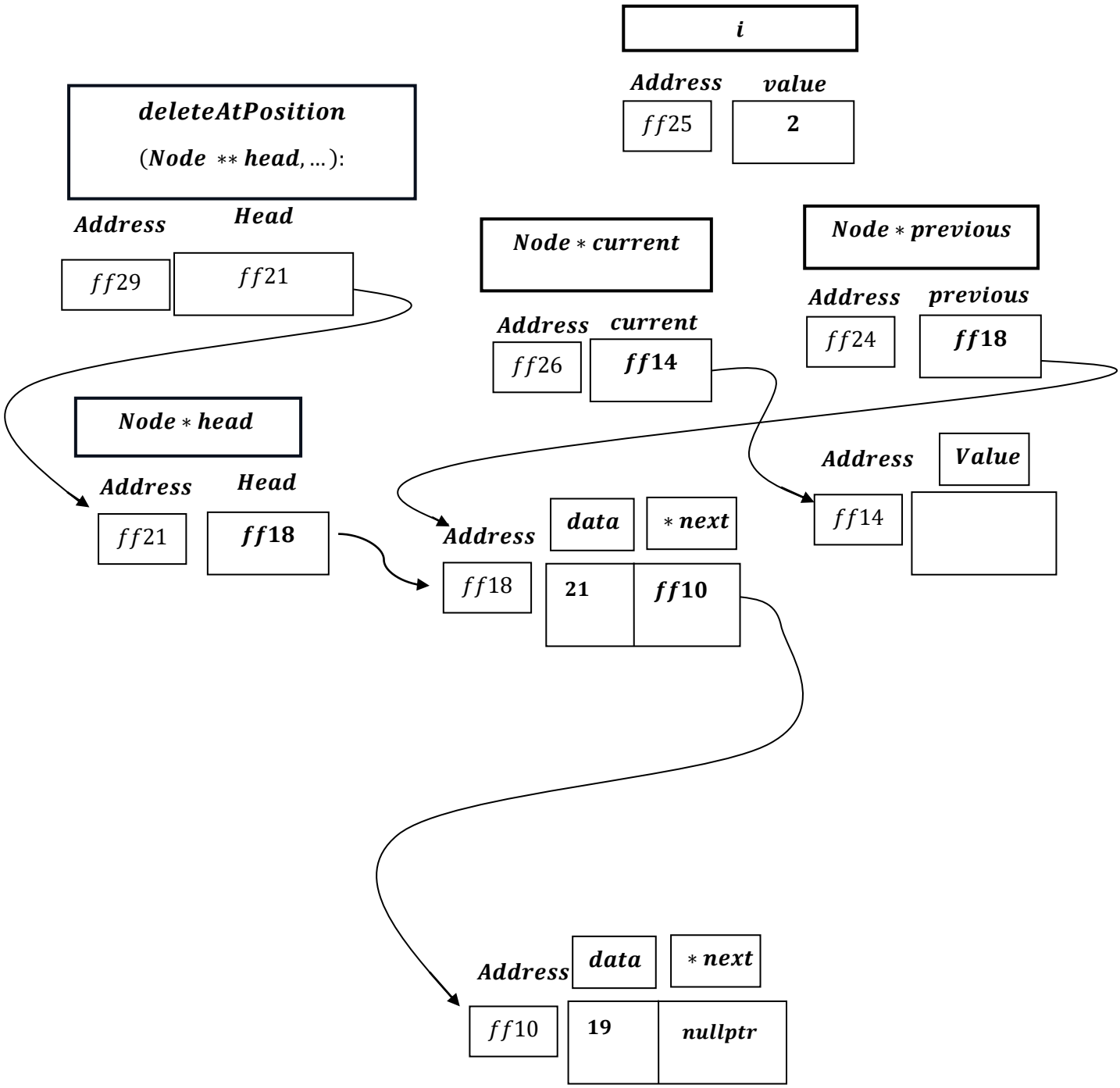


Now we have to free current, which means memory pointed by current pointer will get free but still current pointer will continue to point to same memory address: $ff14$.

As soon as we free the memory pointed to by current, both the data and the next pointer associated with that node are deallocated. The $free(current)$ *call releases the memory,*

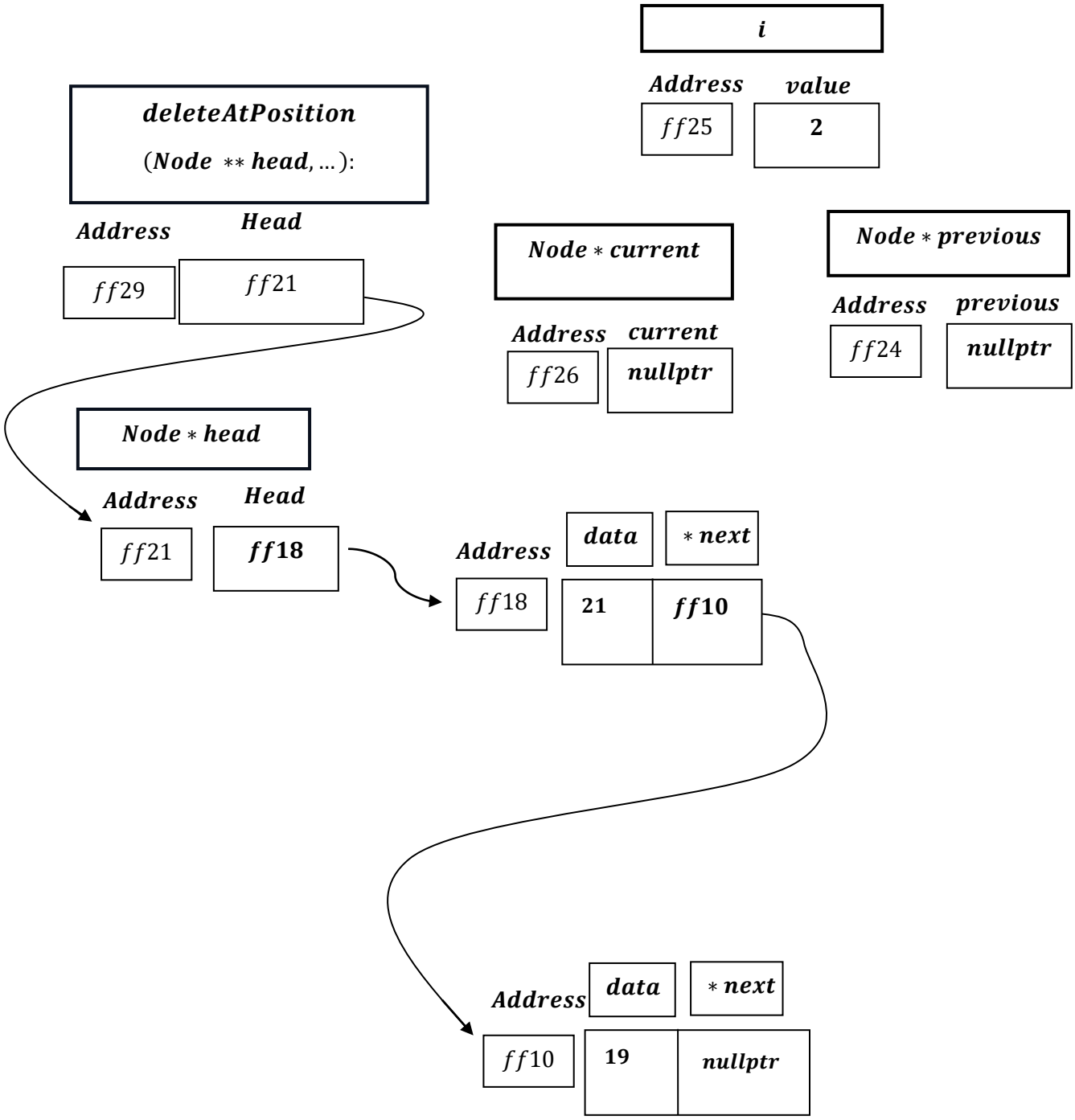
making it available for reuse, but it also means that the entire node (including data and next pointer) is no longer accessible.

```
free(current);
```



Now, to avoid dangling pointers , we assign:

```
current = nullptr;
previous = nullptr;
```

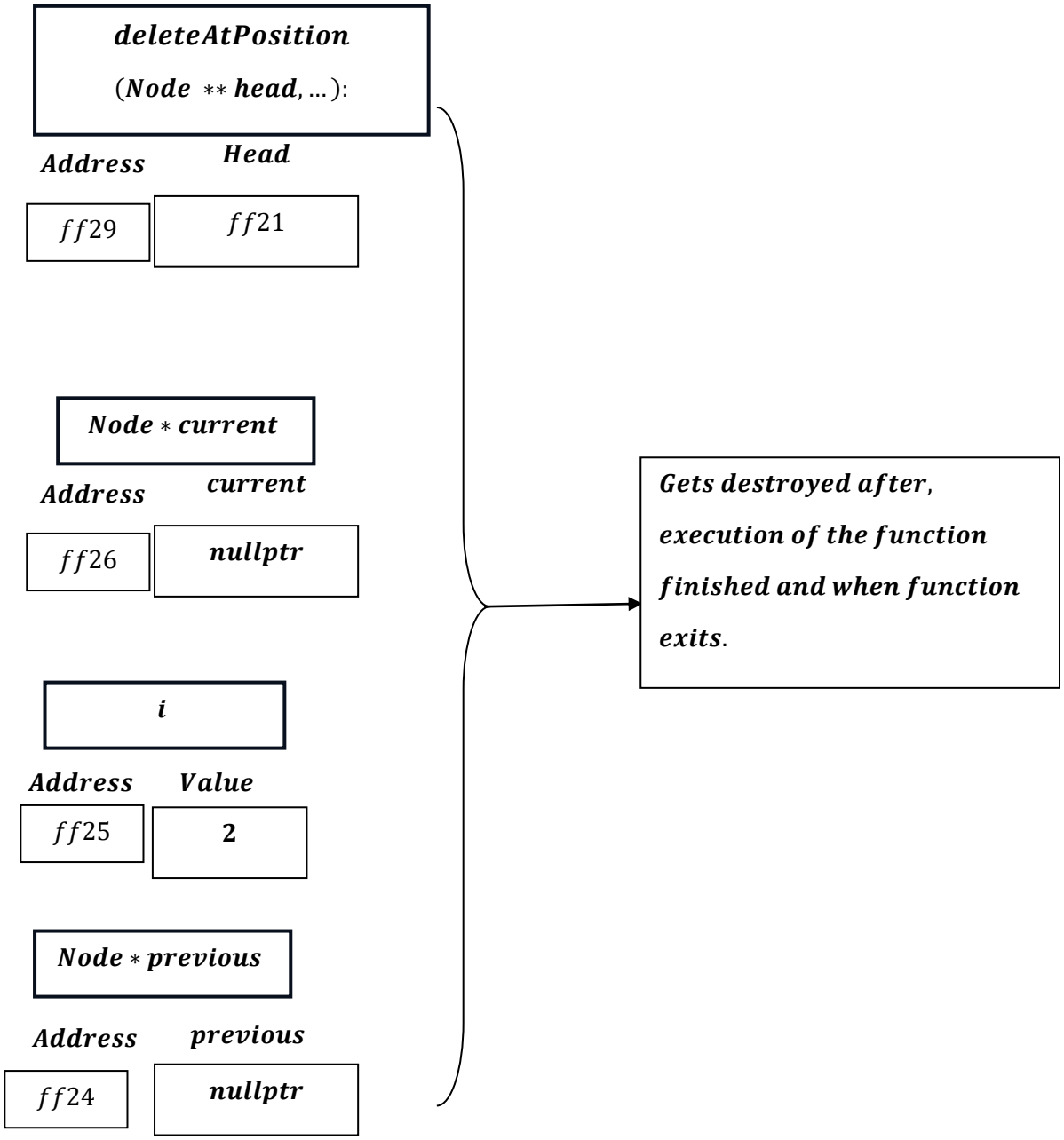


After the process have finished , we print a message about successful deletion of

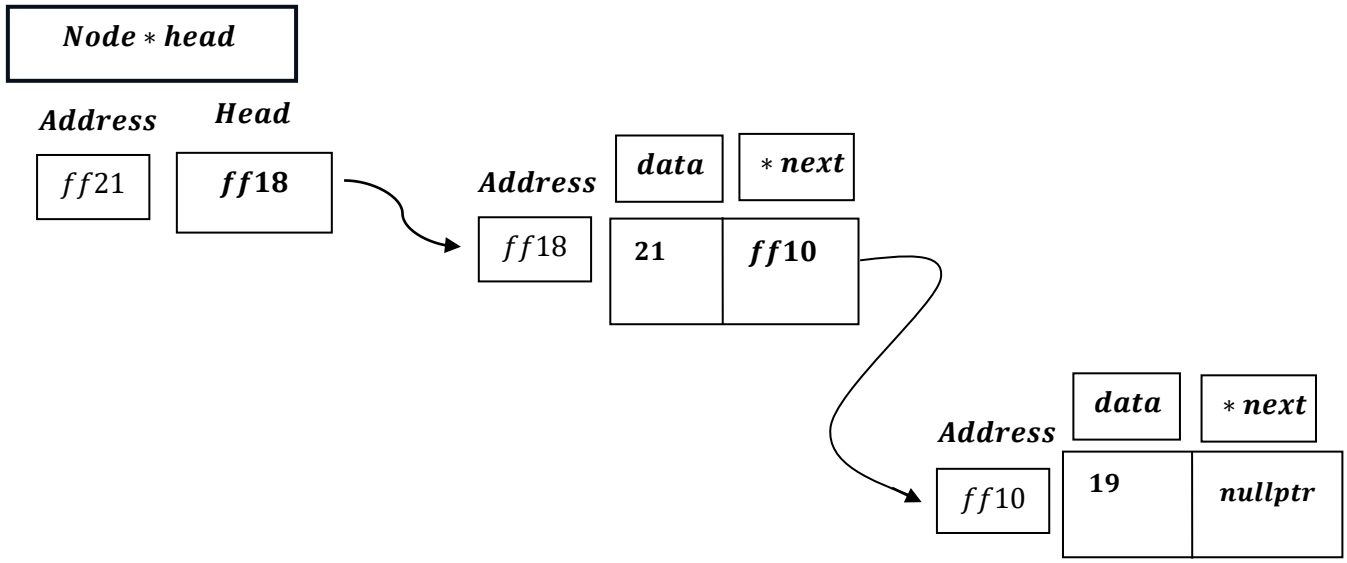
node.

```
cout << "Node is deleted successfully." << endl;
```

After the function module ends , all temporary pointer variables created gets destroyed, i.e.



Hence we are left with:



Step By Step Delete At A Position Explanation:

– *If the head is `nullptr`, the list is empty. Print an error message and return.*

– *If the position is less than 1, it's invalid. Print an error message and return.*

– *`current` points to the head of the list.*

– *`previous` is initialized to `nullptr`.*

– *`i` is initialized to 1 to keep track of the current position.*

– *Traverse the list until you reach the desired position or the end of the list.*

– *Update `previous` to the current node and `current` to the next node in each iteration.*

– Increment `i` to keep track of the current position.

– If `current` is `nullptr`, the position is out of bounds.
Print an error message and return.

– If `previous` is `nullptr`, it means the node to be deleted is the head node.
Update the head to the next node.

– Otherwise, update the `next` pointer of the `previous` node to skip
the `current` node.

– Print the data of the deleted node.

– Set the `next` pointer of the `current` node to `nullptr`.

– Free the memory allocated for the `current` node.

– Set `current` and `previous` to `nullptr` to avoid dangling pointers.

– Print a success message.

The function `deleteAtPosition` deletes a node at a specified position
in a singly linked list. It handles edge cases such as an empty list,
invalid positions, and deleting the head node. The function traverses
the list to find the node at the given position, updates the pointers to
remove the node from the list, and frees the memory allocated for the
node.

Time Complexity of Delete At Position

```
void deleteAtPosition(Node **head, int position)
{
    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return;
    }
    if (position < 1)
    {
        cout << "Error: Invalid position." << endl;
        return;
    }
    else
    {
        Node *current = *head;
        Node *previous = nullptr;
        int i = 1;

        while (i < position && current != nullptr)
        {
            previous = current;
            current = current->next;
            i++;
        }

        if (current == nullptr)
        {
            cout << "Error: Invalid position." << endl;
            return;
        }
```

$O(1)$

$O(1)$

$O(1)$

$O(1)$

$O(1)$

$\rightarrow$ *runs* `$n-1$` *times at worst condition.*

$\rightarrow 1+1+\cdots$ *to* $n-1$ *times* $= O(n)$.

$\rightarrow 1+1+\cdots$ *to* $n-1$ *times* $= O(n)$.

$\rightarrow 1+1+\cdots$ *to* $n-1$ *times* $= O(n)$.

$O(1)$

```
        else
        {
            if (previous == nullptr)
            {
                *head = current->next;
            }

            else
            {
                previous->next = current->next;
            }
          cout << "Deleted node: " << current->data << endl;
          current->next = nullptr;

          free(current);

         current = nullptr;

          previous = nullptr;

        cout << "Node is deleted successfully." << endl;
        }
    }
}
```

$\rightarrow O(1)$

$\rightarrow O(1)$

$\rightarrow O(1)$

$\rightarrow O(1)$

$\rightarrow O(1)$

$\rightarrow O(1)$

$\rightarrow O(1)$

$\rightarrow O(1)$

$\rightarrow O(1)$

There are three case scenarios:

$\rightarrow$ **1.** ***Position is*** **1** ***, i.e. we want to delete data of head node of list .*** **(*Best Case*)**

$\rightarrow$ **2.** ***Position is*** N ***, i.e. we want to delete data at end of list .*** **(*Worst Case*)**

$\rightarrow$ **3.** ***Deleted position from*** **2** ***to*** N***, while loop runs from*** **1** ***to*** $N-1$.

While if position is **1** ***, it takes constant amount of time :*** $O(1)$.

And when the position is uniformly distributed from **1** ***to*** N.

(*Average Case i.e. Undetermined Position*).*That is we donot whether the*

$position\ will\ be\ 1\ or\ 2\ or\ N, where\ Node\ is\ to\ be\ deleted.$

$1.\ When\ Position\ is\ 1(Best\ Case)$

$1. If\ head\ is\ nullptr\ , then\ the\ list\ is\ empty\ , this\ check\ takes\ constant\ time\ along\ with\ priting\ statement\ and\ return\ i.e.\ O(1).$

$2. If\ position\ is\ less\ than\ 1\ , this\ check\ constant\ time\ along\ with\ printing\ statement\ and\ return\ , i.e. O(1).$

$3. As\ position\ is\ 1\ i.e. not\ less\ than\ 1\ , it\ moves\ to\ else\ part\ , assigns\ head\ pointer's\ value\ to\ current\ pointer\ and\ this\ assignment\ takes\ constant\ time\ only\ i.e. O(1).$

$4. Temporary\ variable\ `i`\ assigned\ to\ 1\ takes\ constant\ amount\ of\ time\ i.e. O(1).$

$5. Now, in\ while\ loop\ , condition\ checks\ i.e. i < position\ or\ 1 < 1\ which\ becomes\ false\ , such\ check\ takes\ costant\ amount\ of\ time : O(1).$

$6. Now\ if\ current\ pointer\ value\ is\ nullptr\ , on\ two\ logical\ scenarios\ when :$
$1)\ List\ is\ empty\ and\ 2)\ Position\ is\ greater\ than\ the\ length\ of\ the\ list\ , so\ when\ current\ pointers\ value\ is\ nullptr\ , if\ statement\ checks\ and\ prints\ `position\ is\ Invalid`\ with\ a\ return\ statement\ , takes\ constant\ amount\ of\ time\ i.e. O(1).$

$7. As\ current\ is\ not\ nullptr\ at\ present\ holding\ head\ pointer's\ value\ , hence\ it\ moves\ to\ else\ part\ and\ checks\ if\ previous\ pointer's\ value\ is\ nullptr\ or\ not.\ As\ list\ have\ a\ single\ head\ node\ and\ current\ pointer\ already\ pointed\ to\ single\ head\ , hence, previous\ pointers\ remains\ nullptr, hence\ head\ pointer\ gets\ assigned\ to\ next\ pointer's\ address\ i.e. nullptr\ through\ current\ pointer\ making\ the\ list\ empty. This\ takes\ constant\ amount\ of\ time\ i.e. O(1).$

8. *Print the data of node which is to get deleted which takes constant amount of time i.e.* $O(1)$.

9. *Next we assign next pointer pointed by current pointer to nullptr as, memory location pointed by current pointer will get deleted , hence to stop next pointer pointing to next node we assign nullptr. This takes constant amount of time i.e.* $O(1)$.

10. *Next, we free memory pointed by the current pointer which takes constant amount time i.e.* $O(1)$.

11. *Now we set current to nullptr to avoid dangling pointer , as after freeing of memory of current pointer , it might point to some arbritrary address of any arbritrary memory location, hence we assign nullptr which takes constant amount of time i.e.* $O(1)$.

12. *Next we assign previous pointer to nullptr to stop previous pointer to point to memory location it was pointing to, and it takes constant amount of time i.e.* $O(1)$.

13. *Now it prints a message of successful deletion which takes constant amount of time i.e.* $O(1)$.

Hence , over all time complexity:

$$O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) = O(1).$$

Or, We can say when we position is 1 *, we get the best time complexity* : $\Omega(1)$.

$\underline{2.\ When\ Position\ is\ N(Worst\ Case)}$

$1. If\ head\ is\ nullptr\ , then\ the\ list\ is\ empty\ , this\ check\ takes\ constant\ time\ along\ with\ priting\ statement\ and\ return\ i.e.\ O(1).$

$2. If\ position\ is\ less\ than\ 1\ , this\ check\ constant\ time\ along\ with\ printing\ statement\ and\ return\ , i.e. O(1).$

$3. As\ position\ is\ 1\ i.e. not\ less\ than\ 1\ , it\ moves\ to\ else\ part\ , assigns\ head\ pointer's\ value\ to\ current\ \ pointer\ and\ this\ assignment\ takes\ constant\ time\ only\ i.e. O(1).$

$4. Temporary\ variable\ `i`\ assigned\ to\ 1\ takes\ constant\ amount\ of\ time\ i.e. O(1).$

$5. Now, in\ while\ loop\ , condition\ checks\ i.e.$

$\rightarrow If\ the\ position\ is\ 1, the\ loop's\ inner\ statement\ runs\ 0\ time.$
$\rightarrow If\ the\ position\ is\ 2, the\ loop's\ inner\ statement\ runs\ 1\ time.$
$\rightarrow If\ the\ position\ is\ 3, the\ loop's\ inner\ statement\ runs\ 2\ time.$
.
.
$\rightarrow If\ the\ position\ is\ N, the\ loop's\ inner\ statement\ runs\ N-1\ time.$

$As, we\ are\ finding\ worst\ case\ complexity\ here\ , hence\ we\ take\ position\ `N`\ ,$
$and\ inner\ loop\ statements:$
$previous\ =\ current;$
$current\ =\ current{-}{>}\ next;$
$i++;$

$Also\ runs\ N-1\ times\ each\ , hence, for\ three\ statements\ we\ can\ say, 3(N-1)$

$and\ applying\ Big\ O\ i.e. O\big(3(N-1)\big) = O(N-1) = O(N)\ amount\ of\ time$
$approx$.

$6. Now\ if\ current\ pointer\ value\ is\ nullptr\ , on\ two\ logical\ scenarios\ when$:
$1)\ List\ is\ empty\ and\ 2)\ Position\ is\ greater\ than\ the\ length\ of\ the\ list\ , so$
$when\ current\ pointers\ value\ is\ nullptr\ , if\ statement\ checks\ and\ prints$
$`position\ is\ Invalid`\ with\ a\ return\ statement\ , takes\ constant\ amount\ of$
$time\ i.e. O(1).$

$7. As\ current\ is\ not\ nullptr\ at\ present\ pointing\ N^{th}\ memory\ address\ , hence$
$it\ moves\ to\ else\ part\ and\ checks\ if\ previous\ pointer's\ value\ is\ nullptr\ or\ not.$
$As\ previous\ pointer\ also\ not\ nullptr\ hence\ check\ takes\ O(1), next\ it\ goes\ to\ else$
$part\ , where, next\ \ pointer\ value\ pointed\ by\ previous\ pointer, gets\ assigned\ to$
$next\ pointer\ value\ pointed\ by\ current\ pointer\ address\ i.e. nullptr$
$making\ the\ next\ pointer\ pointed\ by\ previous\ pointer\ to\ stop\ pointing$
$further\ to\ any\ address\ and\ it\ takes\ constant\ amount\ of\ time\ i.e. O(1).$

$8. Print\ the\ data\ of\ node\ which\ is\ to\ get\ deleted\ which\ takes\ constant$
$amount\ of\ time\ i.e. O(1).$

$9. Next\ we\ assign\ next\ pointer\ pointed\ by\ current\ pointer\ to\ nullptr\ as,$
$memory\ location\ pointed\ by\ current\ pointer\ will\ get\ deleted\ , hence\ to$
$stop\ next\ pointer\ pointing\ to\ next\ node\ we\ assign\ nullptr. This\ takes$
$constant\ amount\ of\ time\ i.e. O(1).$

$10. Next, we\ free\ memory\ pointed\ by\ the\ current\ pointer\ which\ takes\ constant$
$amount\ time\ i.e. O(1).$

$11.$ $Now\ we\ set\ current\ to\ nullptr\ to\ avoid\ dangling\ pointer\ ,as\ after\ freeing\ of\ memory\ of\ current\ pointer\ ,it\ might\ point\ to\ some\ arbritrary\ address\ of\ any\ arbritrary\ memory\ location, hence\ we\ assign\ nullptr\ which\ takes\ constant\ amount\ of\ time\ i.e.\ O(1).$

$Hence\ ,over\ all\ time\ complexity:$

$O(1) + O(1) + O(1) + O(1) + O(N) + O(1) + \big(O(1) + O(1)\big) + O(1) + O(1) + O(1)$
$+O(1) + O(1) + O(1) = O(N).$

$Or, We\ can\ say\ when\ we\ position\ is\ 1\ ,we\ get\ the\ best\ time\ complexity:\ O(N).$

$3.\ Deleted\ position\ from\ 2\ to\ N, while\ loop\ runs\ from\ 1\ to\ N-1.$
$And\ when\ the\ position\ is\ uniformly\ distributed\ from\ 1\ to\ N:$
$(Average\ Case\ i.e.\ Undetermined\ Position)$

$\rightarrow If\ the\ position\ is\ 1, the\ loop's\ inner\ statement\ runs\ 0\ time.$
$\rightarrow If\ the\ position\ is\ 2, the\ loop's\ inner\ statement\ runs\ 1\ time.$
$\rightarrow If\ the\ position\ is\ 3, the\ loop's\ inner\ statement\ runs\ 2\ time.$
.
.
$\rightarrow If\ the\ position\ is\ (N), the\ loop's\ inner\ statement\ runs\ (N-1)\ times.$
$As\ when\ i = N < N\ becomes\ false\ ,hence\ no\ execution\ of\ loop's\ inner\ statements.$

$$Hence\ ,\sum_{i=1}^{N-1} i = \frac{N(N-1)}{2}.$$

$Now, as\ we\ take\ all\ the\ possibilites\ then\ , we\ take\ the\ best\ case\ also\ i.e.$
$O(1), when\ position = 1\ . Considering\ `T(P)`\ is\ time\ complexity\ of$
$position\ `P`\ .$

$$T(P) = \begin{cases} 1 & , when\ P = 1. \\ P - 1 & , when\ P = 2, 3, 4, .., N. \end{cases}$$

$Now, If\ there\ is\ `N`\ no. of\ nodes\ there\ we\ can\ delete\ data\ at\ N\ positions.$

$For\ position\ 1, the\ probability\ of\ deletion\ is \frac{1}{N}.$
$For\ position\ 2, the\ probability\ of\ deletion\ is \frac{1}{N}.$
.
.
$For\ position\ N, the\ probability\ of\ deletion\ is \frac{1}{N}.$

$Hence\ probability\ of\ deletion\ will\ always\ remain\ \frac{1}{N}.$

$\therefore\ Average\ Time\ Complexity\ is:\ probability\ of\ deletion \times (number\ of$
$possible\ iterations + constant\ time\ complexities\ for\ left\ over\ positions)\ .$

$$= \left(\frac{1}{N} \times 1 + \frac{1}{N} \times 2 + \cdots + \frac{1}{N} \times N - 1\right) + \frac{1}{N} \times 1$$

$$= \frac{1}{N} \times \{(1 + 2 + 3 + \cdots + N - 1) + 1\}$$

$$= \frac{1}{N} \times \left\{ \sum_{i=1}^{N-1} i + 1 \right\}$$

$$= \frac{1}{N} \times \left(\frac{N(N-1)}{2} + 1 \right)$$

$$= \frac{N-1}{2} + \frac{1}{N}$$

Appplying Big Theta in the equation:

$$= \Theta\left(\frac{N-1}{2}\right) + \Theta\left(\frac{1}{N}\right)$$

Now, taking out the dominant term of numerator and denominator:

$$= \Theta\left(\frac{N}{2}\right) + \Theta\left(\frac{1}{N}\right)$$

$$= \frac{1}{2} \times \Theta(N) + \Theta\left(\frac{1}{N}\right)$$

Now removing the constant `$\frac{1}{2}$` ***we get***:

$$= \Theta(N) + \Theta\left(\frac{1}{N}\right)$$

When combining two Big Theta Θ **terms , largest growth rate is the result**:

$\rightarrow$ $\Theta(N)$ ***grows linearly with*** `$N$` .

$\rightarrow \Theta\left(\frac{1}{N}\right)$ *decreases to a constant , i.e. when N grows larger ,* $\frac{1}{N}$ *approaches to* 0.

This we can prove by limit and continuity also:

$$\lim_{n\to\infty}\left(\frac{1}{n}\right) \Rightarrow \frac{1}{\infty} = 0 \left[\frac{c}{\infty} = 0 \text{ , where } c \text{ is constant}\right].$$

As $\Theta(N)'s$ *growth rate is higher , therefore , average case is* $\Theta(N)$.

Hence, Average Time Complexity is : $\Theta(N)$.

Note , by relation: $\Omega \leq \Theta \leq O$ *, we get*:

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq \frac{N-1}{2} + \frac{1}{N} \leq c_2 \times N$$

Average Case Time Complexity[*Most Simplest Alternative Apporach*]:

(*Simplest approach*)

For position 1, *the probability of deletion is* $\frac{1}{N}$.

For position 2, *the probability of deletion is* $\frac{1}{N}$.

.

.

For position N, *the probability of deletion is* $\frac{1}{N}$.

Hence probability of deletion will always remain $\frac{1}{N}$, *for position* 1 *to* N.

$$\textit{Average case complexity}: \textit{Probability of deletion} \times \begin{pmatrix} \textit{Summation} \\ \textit{of number of} \\ \textit{Position.} \end{pmatrix}$$

$$= \frac{1}{N} \times \left(\sum_{i=1}^{N} i \right)$$

$$= \frac{1}{N} \times \left(\frac{N(N+1)}{2} \right)$$

$$= \frac{N+1}{2}$$

$$or, \frac{N}{2} + \frac{1}{2}$$

Appplying Big Theta in the equation:

$$= \Theta\left(\frac{N}{2}\right) + \Theta\left(\frac{1}{2}\right)$$

Now removing the constant `$\frac{1}{2}$` ***we get***:

$$= \Theta(N) + \Theta(1)$$

$$= \Theta(N).$$

$$\textit{Hence, Average Time Complexity is}: \Theta\left(\frac{N+1}{2}\right) \textit{ or } \Theta(N).$$

$\boldsymbol{Now,}$

$\boldsymbol{Note\ , by\ relation: \Omega \leq \Theta \leq O\ , we\ get:}$

$$\Longrightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Longrightarrow c_1 \times 1 \leq \frac{N+1}{2} \leq c_2 \times N$$

Singly Linked List	*Cases*	*Time Complexity*
Delete At Position	1. *Best Case*	$\Omega(1)$
	2. *Worst Case*	$O(n)$
	3. *Average Case*	$\Theta(n)$

****